CENTRO PAULA SOUZA
FATEC SOROCABA
Análise e Desenvolvimento de Sistemas

Gabriel da Silva, Giovanni Camargo Maruccio e Luís Felipe Glauser Lucas

SISTEMA INTELIGENTE DE MONITORAMENTO DE ESTUFA

SOROCABA - SP
2026

SUMÁRIO

1. INTRODUÇÃO.....3
1.1 Tema Escolhido: Agricultura de Precisão.....3
1.2 Variáveis Monitoradas e Faixas Ideais.....4
2. DIAGRAMA DA ARQUITETURA.....5
2.1 Componentes da Infraestrutura.....5
3. EXPLICAÇÃO DA STACK MING.....7
3.1 IoT — Código do Simulador (Google Colab).....7
Dados Enviados — Descrição dos Campos.....7
3.2 Node-RED — Pipeline de Dados.....8
Configuração do Nó InfluxDB Out.....8
3.3 InfluxDB — Banco de Séries Temporais.....9
3.4 Grafana — Dashboard em Tempo Real.....9
4. EXPLICAÇÃO DA CAMADA DE APLICAÇÃO (STACK WEB).....10
4.1 Backend — Regras de Negócio.....10
Limiares (Thresholds) Implementados.....10
API REST — Endpoints Documentados.....10
4.2 MySQL — Esquema do Banco de Dados (Schema).....11
4.3 Frontend — Integração e Interface.....12
Componentes e Integração.....12
5. GUIA DE EXECUÇÃO DO PROJETO.....13
5.1 Pré-requisitos.....13
5.2 Configuração da EC2 — Stack MING.....13
5.3 Configuração Local — Stack Web.....13
1. Variáveis de Ambiente do Backend.....13
5.4 Manutenção: Atualização de IPs.....14
6. ESTRUTURA DO REPOSITÓRIO.....16

1. INTRODUÇÃO

O projeto realiza o monitoramento em tempo real do ambiente de uma **estufa agrícola**, coletando dados de temperatura, umidade, luminosidade e qualidade do ar. O objetivo é garantir que as plantas permaneçam em condições ideais para o crescimento e a produção, contando com alertas automáticos sempre que uma variável ultrapassa a faixa de segurança.

O Problema: Pequenas variações climáticas podem comprometer toda uma produção vegetal. Um pico de temperatura de poucas horas é suficiente para matar mudas que levaram semanas para se desenvolver, enquanto a umidade excessiva favorece a proliferação de fungos. Com o monitoramento contínuo, o produtor pode agir preventivamente, antes que o dano ocorra.

1.1 Tema Escolhido: Agricultura de Precisão

- **Monitoramento Contínuo:** Acompanhamento de quatro variáveis ambientais críticas.
- **Alertas Automáticos:** Notificações imediatas baseadas em *thresholds* (limiares) agronômicos.
- **Histórico Consolidado:** Armazenamento de dados para análise de padrões e tomada de decisão estratégica.
- **Dashboard em Tempo Real:** Interface intuitiva para visualização remota dos dados pelo produtor.

1.2 Variáveis Monitoradas e Faixas Ideais

Variável	Faixa Ideal	Alerta	Crítico	Impacto (Fora da Faixa)
Temperatura	18 a 28 °C	> 28 °C	> 35 °C	Estresse térmico, queima de folhas e morte de mudas.
Umidade	50 a 70%	> 70%	> 80%	Surgimento de fungos (*Botrytis*) e podridão de raízes.
Luminosidade	200 a 800 lux	> 1500 lux	> 1800 lux	Fotoinibição (excesso) ou etiolamento (falta de luz).
Qualidade do Ar	400 a 600 ppm	> 600 ppm	> 800 ppm	

2. DIAGRAMA DA ARQUITETURA

A arquitetura é dividida em duas camadas complementares: a **Stack MING** (Mosquitto, InfluxDB, Node-RED e Grafana) para a ingestão e visualização de dados brutos, e a **Stack Web** para a lógica de aplicação e entrega de valor ao produtor final.

2.1 Componentes da Infraestrutura

Componente	Tecnologia	Porta	Função
Simulador IoT	Google Colab + Python	—	Simula o comportamento dos sensores ESP32 via protocolo MQTT.
MQTT Broker	Eclipse Mosquitto	1883	Centraliza e distribui as mensagens enviadas pelos sensores.
Orquestrador	Node-RED	8082	Processa o fluxo de dados MQTT e realiza a persistência no InfluxDB.
Time Series DB	InfluxDB 2.x	8083	Banco de dados temporal para armazenamento de leituras brutas.
Monitoramento	Grafana	8084	*Dashboard* técnico em tempo real conectado ao InfluxDB.
API REST	Node.js + Express	8080	Camada de backend que consolida dados e gerencia lógica de alertas.
Banco Relacional	MySQL 8.4	3307	Armazena dados estruturados, usuários e histórico de alertas.

Componente	Tecnologia	Porta	Função
Interface (Web)	React + TypeScript	3000	Painel do produtor com atualização automática (*polling*) a cada 5s.

3. EXPLICAÇÃO DA STACK MING

A **Stack MING** (Mosquitto, InfluxDB, Node-RED e Grafana) é responsável por toda a ingestão, armazenamento e visualização dos dados brutos dos sensores da estufa. Toda a infraestrutura é executada em uma instância **AWS EC2** via **Docker Compose**, garantindo isolamento e facilidade de deploy.

3.1 IoT — Código do Simulador (Google Colab)

Devido a limitações técnicas com o simulador Wokwi no ambiente de desenvolvimento, utilizou-se o **Google Colab** para emular o comportamento de um **ESP32**. O script Python pública objetos JSON via protocolo **MQTT** a cada 5 segundos no tópico `fatec/pbl01`:

```
# Instalação da biblioteca cliente
!pip install paho-mqtt

import paho.mqtt.client as mqtt
import json, time, random
from datetime import datetime

MQTT_BROKER = "IP_DA_EC2"
MQTT_TOPIC  = "fatec/pbl01"

client = mqtt.Client()
client.connect(MQTT_BROKER, 1883, 60)

while True:
    dados = {
        "temperatura":  round(random.uniform(20.0, 30.0), 2),
        "umidade":      round(random.uniform(40.0, 70.0), 2),
        "luminosidade": round(random.uniform(100, 1000), 2),
        "qualidade_ar": random.randint(100, 500),
        "timestamp":    datetime.now().strftime("%Y%m%d%H%M%S")
    }
    client.publish(MQTT_TOPIC, json.dumps(dados))
    print(f"Publicado: {dados}")
    time.sleep(5)
```

Dados Enviados — Descrição dos Campos

Campo JSON	Tipo	Faixa Simulada	Descrição
temperatura	float	20.0 a 30.0 °C	Temperatura do ar interna.
umidade	float	40.0 a 70.0%	Umidade relativa do ar.

luminosidade	float	100 a 1000 lux	Intensidade de luz para fotossíntese.
qualidade_ar	int	100 a 500 ppm	Concentração de CO_2.
timestamp	string	YYYYMMDDHHmmss	Carimbo de tempo da coleta.

3.2 Node-RED — Pipeline de Dados

O **Node-RED** atua como a camada de orquestração. Ele consome as mensagens do tópico fatec/pbl01, processa o payload e realiza a persistência no **InfluxDB**. O arquivo de configuração está disponível em /nodered/flows.json.

Estrutura do Fluxo:

1. **MQTT In:** Assina o tópico no broker Mosquitto (porta 1883).
2. **JSON:** Realiza o *parsing* da string recebida para objeto JavaScript.
3. **Function (opcional):** Aplica validações e formatação de campos.
4. **InfluxDB Out:** Grava os dados no bucket iot-sensors.

Configuração do Nó InfluxDB Out

Parâmetro	**Valor Configurado**
Measurement	iot-sensors
Organization	my-org
Bucket	iot-sensors
URL Interna	http://influxdb:8086
Precisão de Tempo	milissegundos (ms)

3.3 InfluxDB — Banco de Séries Temporais

O **InfluxDB 2.x** armazena as leituras brutas de forma otimizada para consultas temporais.

- **Organização:** my-org
- **Retenção:** Infinita (ajustável conforme necessidade de armazenamento).
- **Linguagem de Consulta:** Flux.
- **Acesso:** Mapeado para a porta externa **8083**.

3.4 Grafana — Dashboard em Tempo Real

O Grafana atua como a interface de monitoramento técnico, conectado ao InfluxDB como fonte de dados (*Data Source*). O painel foi projetado para oferecer uma visão imediata e analítica da telemetria, apresentando:

- **Gráficos de Linhas:** Histórico detalhado das quatro variáveis ambientais (últimos 15 minutos).
- **Gauges de Temperatura:** Visualização em tempo real com escala de cores dinâmica, facilitando a identificação visual de faixas seguras ou críticas.
- **Atualização Automática:** *Refresh* de dados configurado para cada 5 segundos, garantindo baixa latência na visualização.
- **Modo Kiosk:** Configuração de interface limpa, otimizada para a exibição integrada dentro do frontend React.

Para viabilizar a integração transparente via *iframe*, o serviço foi configurado com as seguintes variáveis de ambiente:

- `GF_AUTH_ANONYMOUS_ENABLED=true`: Habilita o acesso de leitura anônima, dispensando login manual para visualização dos dados.
- `GF_SECURITY_ALLOW_EMBEDDING=true`: Permite que os painéis sejam incorporados em outras aplicações, removendo restrições de segurança de renderização em quadros (*frames*).

4. EXPLICAÇÃO DA CAMADA DE APLICAÇÃO (STACK WEB)

A **Stack Web** atua como o cérebro analítico do sistema. Ela consome os dados brutos do InfluxDB, consolida-os em intervalos de 5 minutos, detecta condições críticas baseadas em regras agronômicas e entrega as informações ao produtor por meio de uma **API REST** e um **Frontend React**.

4.1 Backend — Regras de Negócio

O backend, desenvolvido em Node.js, gerencia um **Job de Consolidação** que é executado a cada 30 segundos. Suas principais funções são:

1. **Agregação:** Consulta o InfluxDB para obter as leituras dos últimos 5 minutos.
2. **Processamento:** Agrupa os dados por sensor_id e métrica, calculando a média, o valor mínimo e o máximo de cada grupo.
3. **Persistência:** Salva os resultados consolidados na tabela metrics_consolidated do MySQL.
4. **Monitoramento de Limiares:** Compara as médias obtidas com os limiares (*thresholds*) agronômicos predefinidos.
5. **Gestão de Incidentes:** Gera alertas automáticos no banco de dados quando um limiar é ultrapassado e atualiza o status dos sensores (Online, Warning ou Offline).

Limiares (*Thresholds*) Implementados

Métrica	Alerta (Warning)	Crítico (Danger)	Unidade
Temperatura	> 28	> 35	°C
Umidade	> 70	> 80	%
Qualidade do Ar (CO_2)	> 600	> 800	ppm
Luminosidade	> 1500	> 1800	lux

API REST — Endpoints Documentados

Método	Rota	Descrição

GET	/api/health	Verifica o status da API e a conexão com o banco de dados.
GET	/api/sensors	Lista sensores, níveis de bateria, sinal e última atividade.
GET	/api/sensors/:id	Detalhes de um sensor específico e suas leituras consolidadas.
PATCH	/api/sensors/:id	Atualiza metadados do sensor (nome ou localização).
GET	/api/metrics	Resumo das métricas da última janela de 5 minutos.
GET	/api/metrics/:m/trend	Série histórica para geração de gráficos (*sparklines*).
GET	/api/alerts	Lista alertas ativos ou históricos (?resolved=false/true).
PATCH	/api/alerts/:id/resolve	Registra a resolução de um alerta após intervenção técnica.

4.2 MySQL — Esquema do Banco de Dados (Schema)

Tabela	**Descrição**
sensors	Cadastro dos dispositivos: ID, nome, localização, status, bateria e sinal.
metrics_consolidated	Dados processados por janela: médias, mínimos, máximos e intervalos.
alerts	Registro de incidentes: UUID, severidade, mensagem e *timestamps* de ocorrência.

4.3 Frontend — Integração e Interface

O frontend em **React + TypeScript** consome a API REST e realiza atualizações automáticas via *polling* através do hook customizado useIoTData. A interface foi projetada para alta legibilidade no campo.

Componentes e Integração

- **StatsBar:** Consome /api/sensors e /api/alerts para exibir o panorama geral da estufa.
- **MetricCard:** Exibe o valor atual da métrica e uma *sparkline* de tendência histórica.
- **SensorTable:** Monitora a saúde do hardware em tempo real.
- **AlertFeed:** Centraliza notificações críticas com ação de resolução imediata.
- **GrafanaDashboard:** Incorpora os gráficos detalhados da Stack MING via iframe.

Código de Integração — Hook useIoTData

```
const POLL_INTERVAL_MS = 5_000; // Atualização a cada 5 segundos

useEffect(() => {
  fetchAll(); // Busca inicial de dados
  const id = setInterval(fetchAll, POLL_INTERVAL_MS);

  return () => clearInterval(id); // Limpeza do intervalo ao desmontar
}, []);

async function fetchAll() {
  // Execução paralela para otimizar a performance
  const [sensors, metrics, alerts] = await Promise.all([
    fetchSensors(), // GET /api/sensors
    fetchMetrics(), // GET /api/metrics
    fetchAlerts(),  // GET /api/alerts
  ]);
  // Atualiza os estados globais da aplicação
}
```

5. GUIA DE EXECUÇÃO DO PROJETO

5.1 Pré-requisitos

Para a correta execução do sistema, certifique-se de possuir os seguintes itens:

- **AWS EC2:** Instância com Ubuntu 22.04, Docker e Docker Compose instalados.
- **Node.js v18+:** Instalado na máquina local (ou na instância de aplicação).
- **Google Colab:** Conta Google para execução do script de simulação IoT.
- **Git:** Para clonagem do repositório.

5.2 Configuração da EC2 — Stack MING

1. **Clonar o repositório:**

```
git clone https://github.com/profAndreSouza/StackMingWeb.git
cd Stacking Web
```

2. **Subir os containers da Stack MING:**

```
docker-compose up -d nodered influxdb grafana mqtt
```

3. **Verificar o status dos serviços:**

```
docker-compose ps # Todos os containers devem apresentar o status "Up" ou "Healthy".
```

5.3 Configuração Local — Stack Web

1. Variáveis de Ambiente do Backend

Crie o arquivo `backend/.env` e configure-o com o IP público da sua instância EC2:

```
INFLUX_URL=http://IP_DA_EC2:8083
INFLUX_TOKEN=seu_token_influxdb
INFLUX_ORG=my-org
INFLUX_BUCKET=iot-sensors
MYSQL_HOST=localhost
MYSQL_PORT=3307
MYSQL_DATABASE=iot_consolidated
MYSQL_USER=iot_user
MYSQL_PASSWORD=iot_pass
USE_MOCK=false
CONSOLIDATION_CRON=*/30 * * * * *
```

```
CONSOLIDATION_WINDOW_MINUTES=5
```

2. Iniciar o Backend:

```
cd backend
npm install
npm run seed   # Executar apenas uma vez para criar tabelas e sensores iniciais
npm start
```

3. Configurar e Iniciar o Frontend:

Crie o arquivo frontend/.env definindo a URL da API:
VITE_API_URL=http://localhost:8080:

```
cd frontend
npm install
npm start
```

4. Iniciar o Simulador de Sensores

Abra o Google Colab e execute o notebook localizado em /iot/simulador_estufa.ipynb.
Atenção: Atualize a variável MQTT_BROKER com o IP da sua EC2 antes de iniciar a execução.

5.4 Manutenção: Atualização de IPs

Como o IP público da AWS EC2 pode ser alterado a cada reinicialização (caso não seja utilizado um Elastic IP), é necessário atualizar os seguintes pontos:

Arquivo / Local	**Campo a Alterar**
backend/.env	INFLUX_URL=http://NOVO_IP:8083
frontend/.env	VITE_API_URL=http://NOVO_IP:8080 (se o frontend estiver na EC2)
frontend/src/pages/GrafanaDashboard.tsx	Atributo src no iframe do Grafana

Google Colab	Célula de configuração: MQTT_BROKER = 'NOVO_IP'

6. ESTRUTURA DO REPOSITÓRIO

O repositório segue a estrutura obrigatória definida no escopo do **PBL1**, organizado de forma modular para separar as responsabilidades de cada camada do sistema (Ingestão, Processamento e Interface):

```
StackMingWeb/
├── /iot
│   └── simulador_estufa.ipynb   # Código Google Colab (Emulador de ESP32)
│
├── /nodered
│   └── flows.json               # Fluxo de dados exportado do Node-RED
│
├── /backend
│   ├── src/
│   │   ├── index.js             # Ponto de entrada (Boot da API)
│   │   ├── app.js               # Configuração Express e Middlewares
│   │   ├── db/
│   │   │   ├── influx.js        # Consultas InfluxDB (Linguagem Flux)
│   │   │   ├── migrate.js       # Script de criação das tabelas MySQL
│   │   │   └── seed.js          # Inserção de dados iniciais (Sensores)
│   │   ├── routes/
│   │   │   ├── sensors.js       # Gerenciamento (CRUD) de sensores
│   │   │   ├── metrics.js       # Endpoints de consulta de métricas
│   │   │   └── alerts.js        # Gerenciamento e resolução de alertas
│   │   └── jobs/
│   │       └── consolidation.js # Job de agregação de dados e detecção de alertas
│   └── package.json
│
├── /frontend
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Dashboard.tsx    # Painel principal de monitoramento IoT
│   │   │   └── GrafanaDashboard.tsx # Visualização técnica (Iframe Grafana)
│   │   ├── components/          # Componentes UI (MetricCard, SensorTable, AlertFeed)
│   │   ├── hooks/
│   │   │   └── useIoTData.ts    # Custom Hook para polling automático (5s)
│   │   └── services/
│   │       └── api.ts           # Cliente de comunicação com o Backend REST
│   └── package.json
│
├── /docs
│   ├── PBL1_Monitoramento_Estufa.docx # Documentação técnica do projeto
│   ├── arquitetura.png          # Diagrama da arquitetura do sistema
│   ├── grafana_dashboard.png    # Captura de tela do dashboard Grafana
│   └── frontend_dashboard.png   # Captura de tela da interface React
│
└── docker-compose.yml           # Orquestração da Stack MING na AWS EC2
```